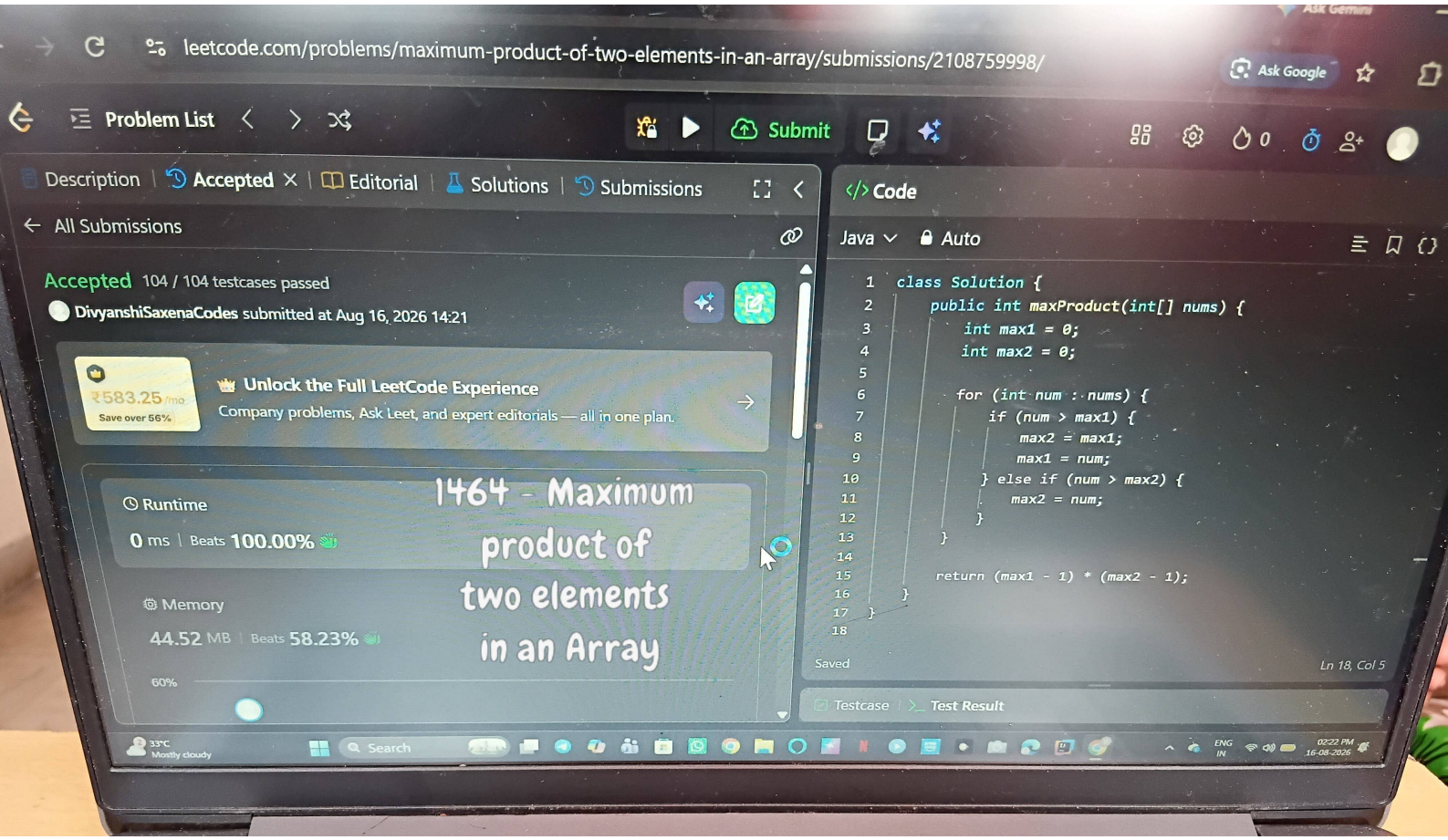
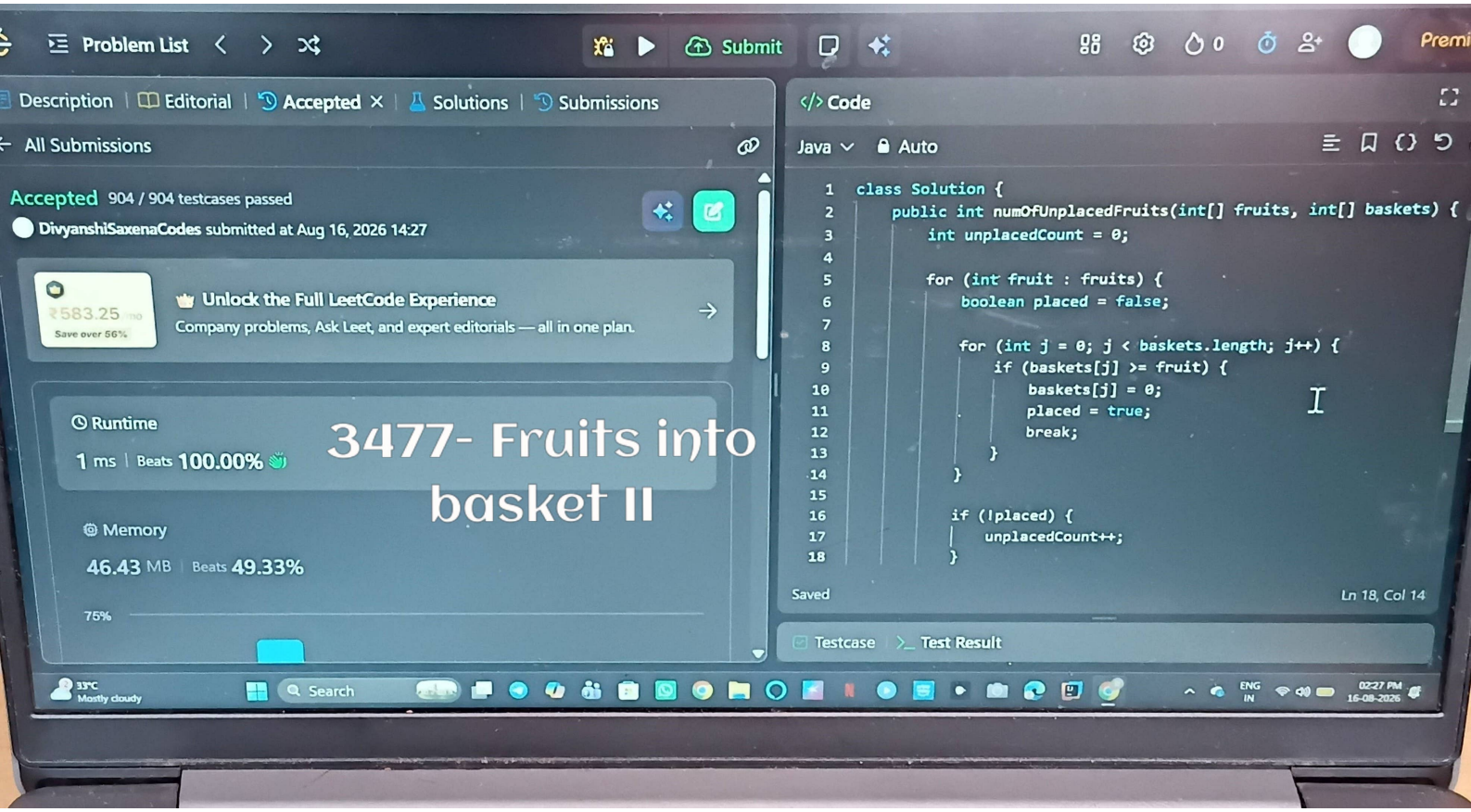
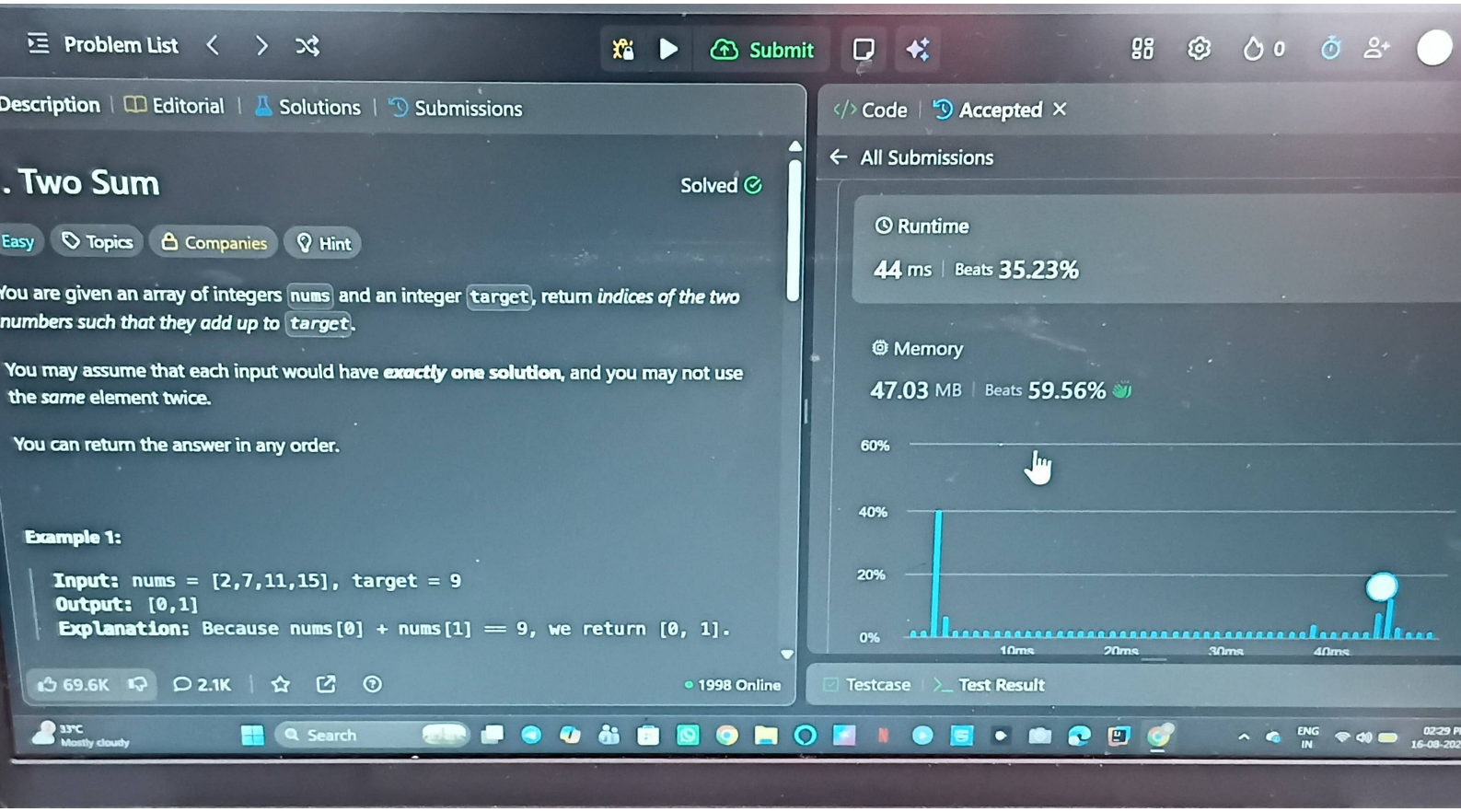
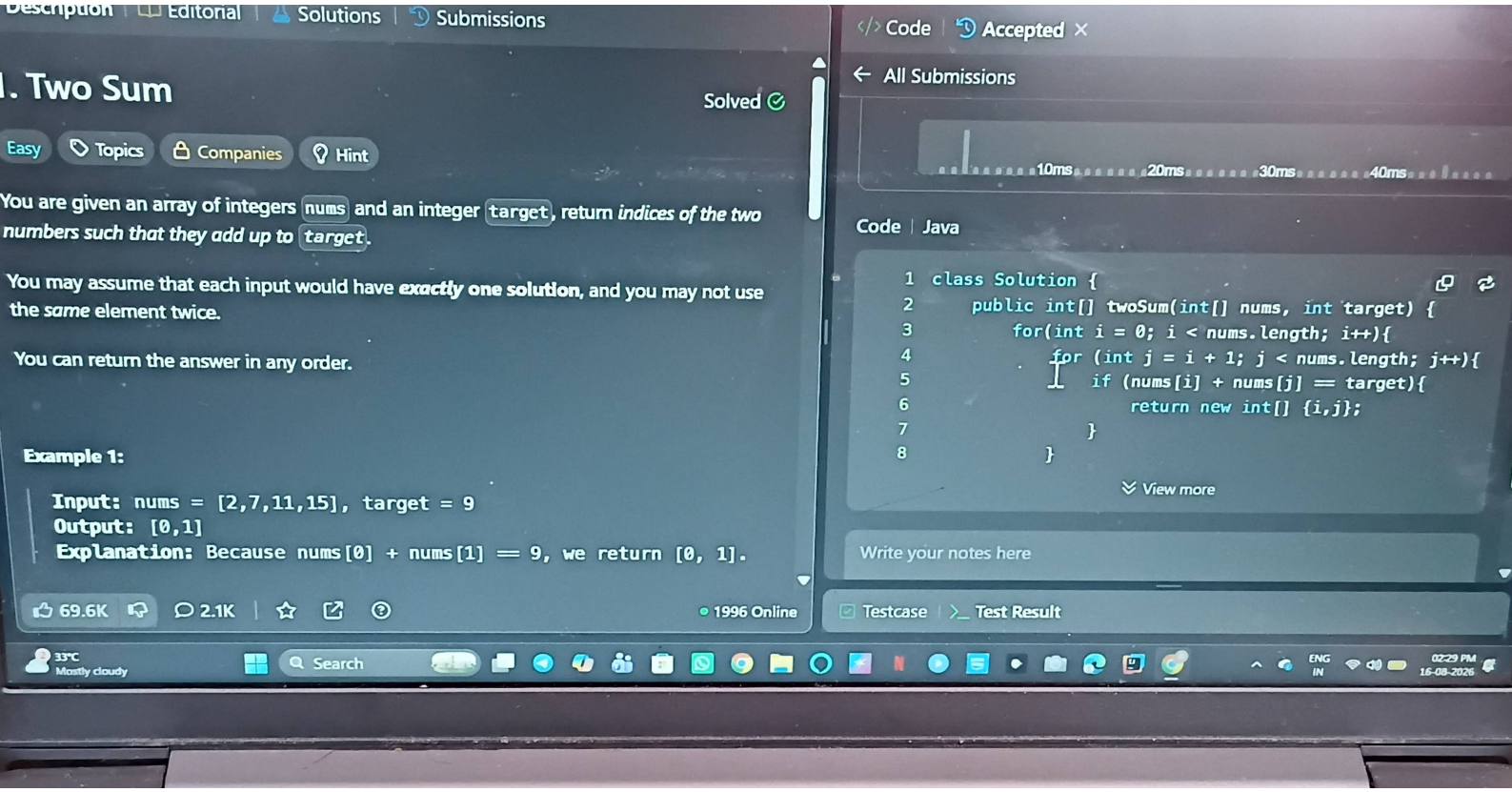
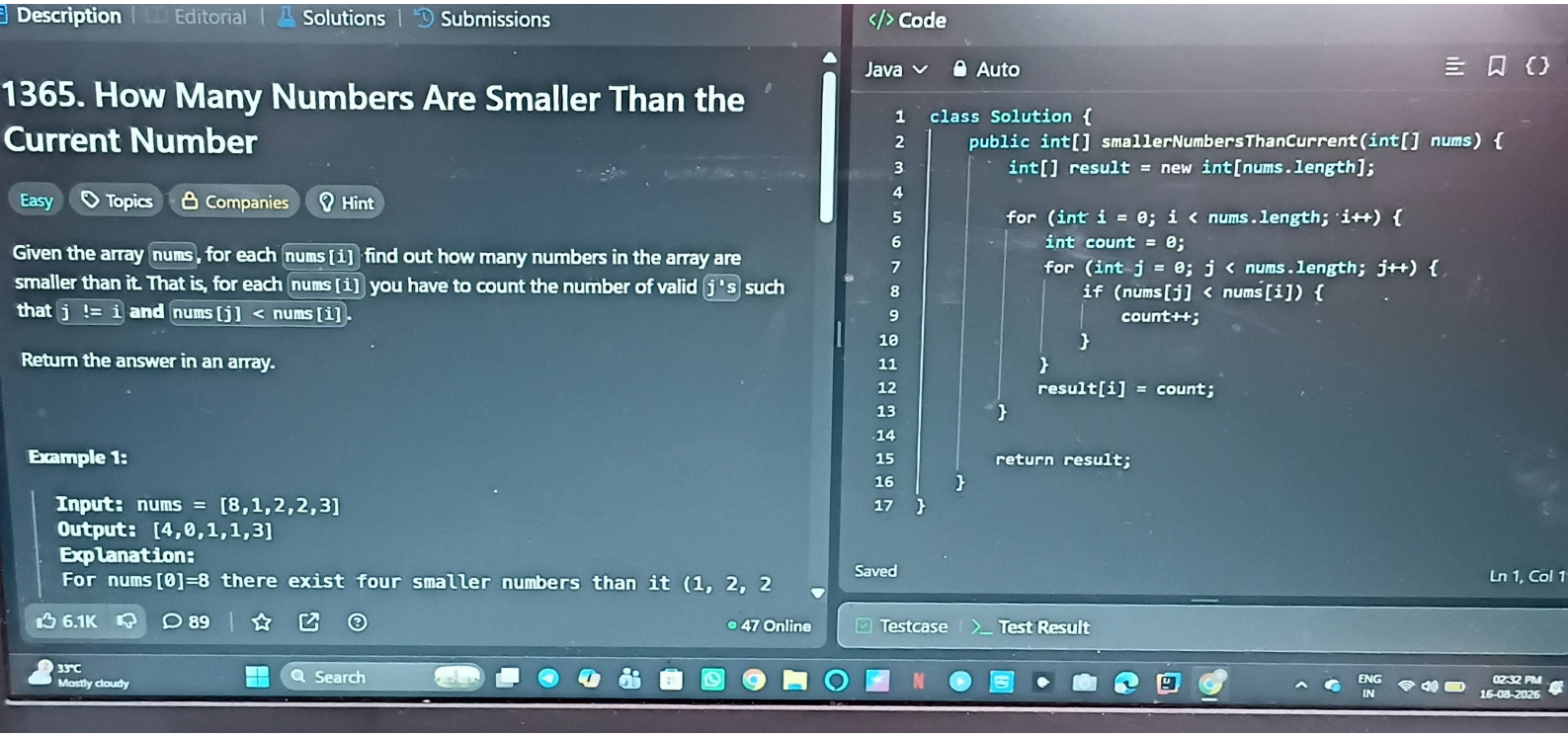
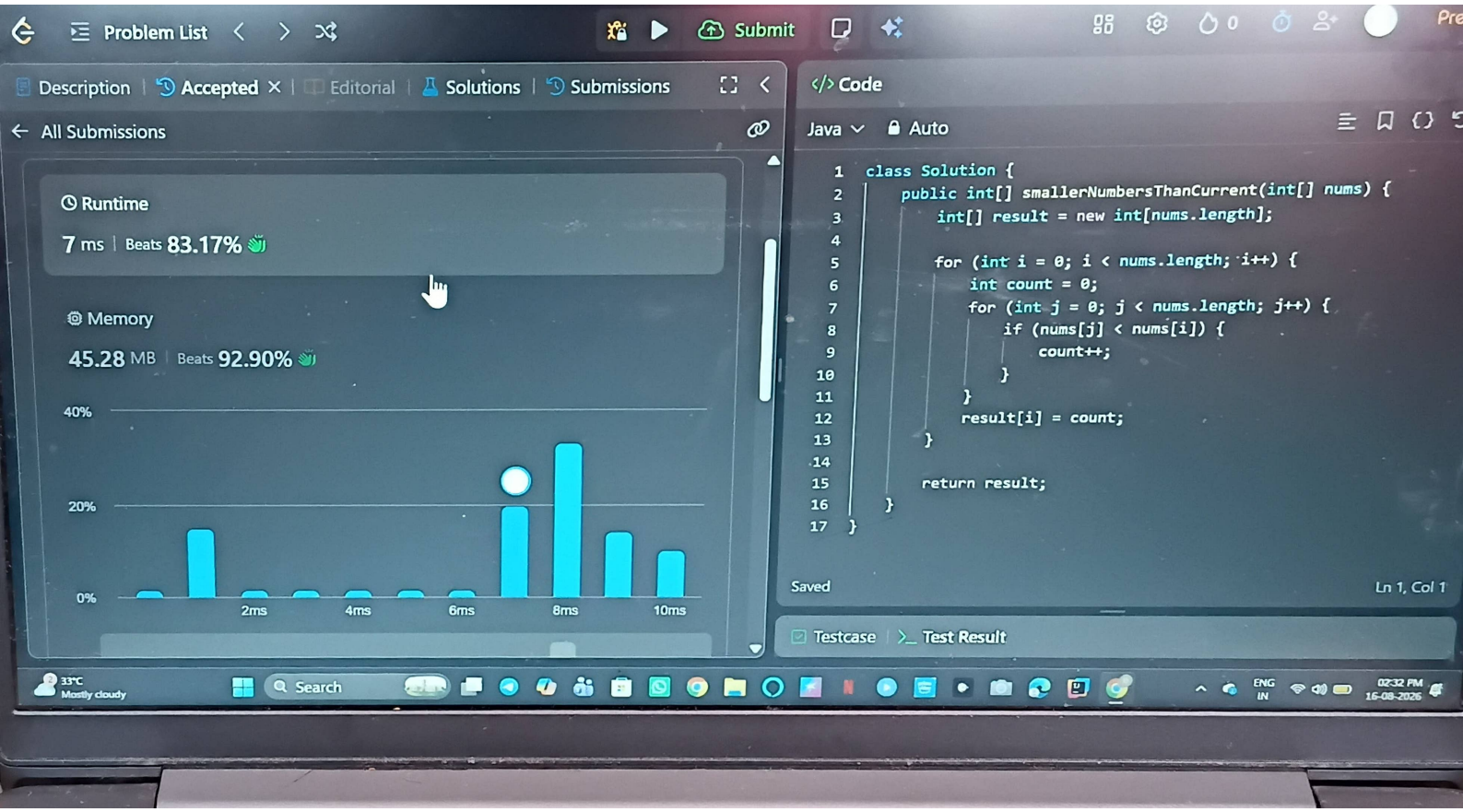

leetcode.com/problems/maximum-product-of-two-elements-in-an-array/submissions/2108759998/
Problem List
Submit
Description | Accepted | Editorial | Solutions | Submissions
All Submissions
Accepted 104 / 104 testcases passed
DivyanshiSaxenaCodes submitted at Aug 16, 2026 14:21
Unlock the Full LeetCode Experience
Company problems, Ask Leet, and expert editorials — all in one plan.
Runtime
0 ms | Beats 100.00%
Memory
44.52 MB | Beats 58.23%
1464 - Maximum product of two elements in an Array
Code
Java
Auto
class Solution {
public int maxProduct(int[] nums) {
int max1 = 0;
int max2 = 0;
for (int num : nums) {
if (num > max1) {
max2 = max1;
max1 = num;
} else if (num > max2) {
max2 = num;
}
}
return (max1 - 1) * (max2 - 1);
}
}
Saved
Ln 18, Col 5
Testcase
Test Result

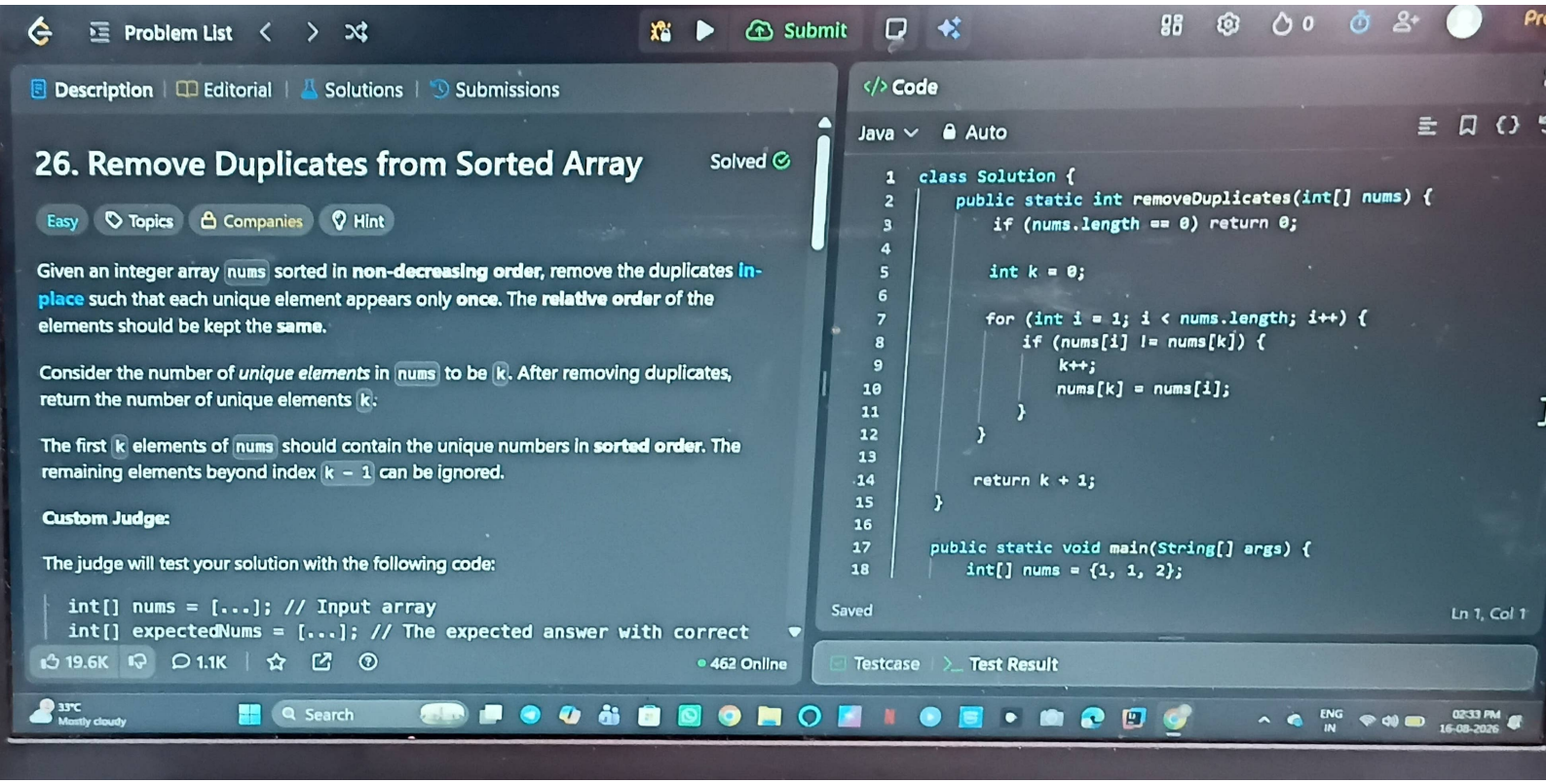

Problem List
Submit
Description | Editorial | Accepted | Solutions | Submissions
All Submissions
Accepted 904 / 904 testcases passed
DivyanshiSaxenaCodes submitted at Aug 16, 2026 14:27
Unlock the Full LeetCode Experience
Company problems, Ask Leet, and expert editorials — all in one plan.
Save over 56%
Runtime
1 ms | Beats 100.00%
Memory
46.43 MB | Beats 49.33%
75%
3477- Fruits into basket II
Code
Java
Auto
class Solution {
public int numOfUnplacedFruits(int[] fruits, int[] baskets) {
int unplacedCount = 0;
for (int fruit : fruits) {
boolean placed = false;
for (int j = 0; j < baskets.length; j++) {
if (baskets[j] >= fruit) {
baskets[j] = 0;
placed = true;
break;
}
}
if (!placed) {
unplacedCount++;
}
Saved
Ln 18, Col 14
Testcase
Test Result
Search

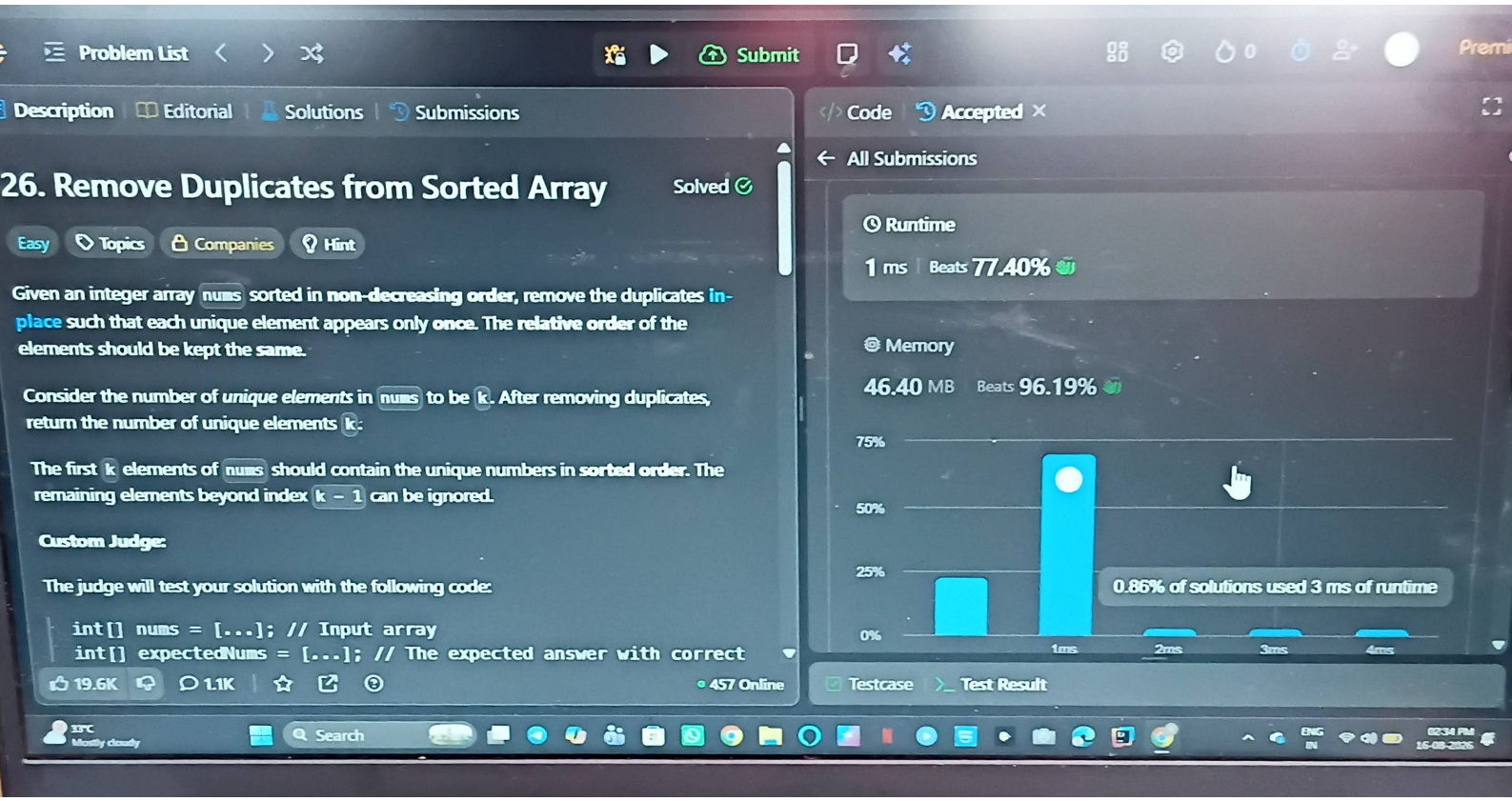

Problem List
Submit
Description | Editorial | Solutions | Submissions
. Two Sum
Solved
Easy
Topics
Companies
Hint
You are given an array of integers nums and an integer target, return indices of the two numbers such that they add up to target.
You may assume that each input would have exactly one solution, and you may not use the same element twice.
You can return the answer in any order.
Example 1:
Input: nums = [2,7,11,15], target = 9
Output: [0,1]
Explanation: Because nums[0] + nums[1] == 9, we return [0, 1].
69.6K
2.1K
1998 Online
Code | Accepted
All Submissions
Runtime
44 ms | Beats 35.23%
Memory
47.03 MB | Beats 59.56%
60%
40%
20%
0%
10ms
20ms
30ms
40ms
Testcase | Test Result
Search

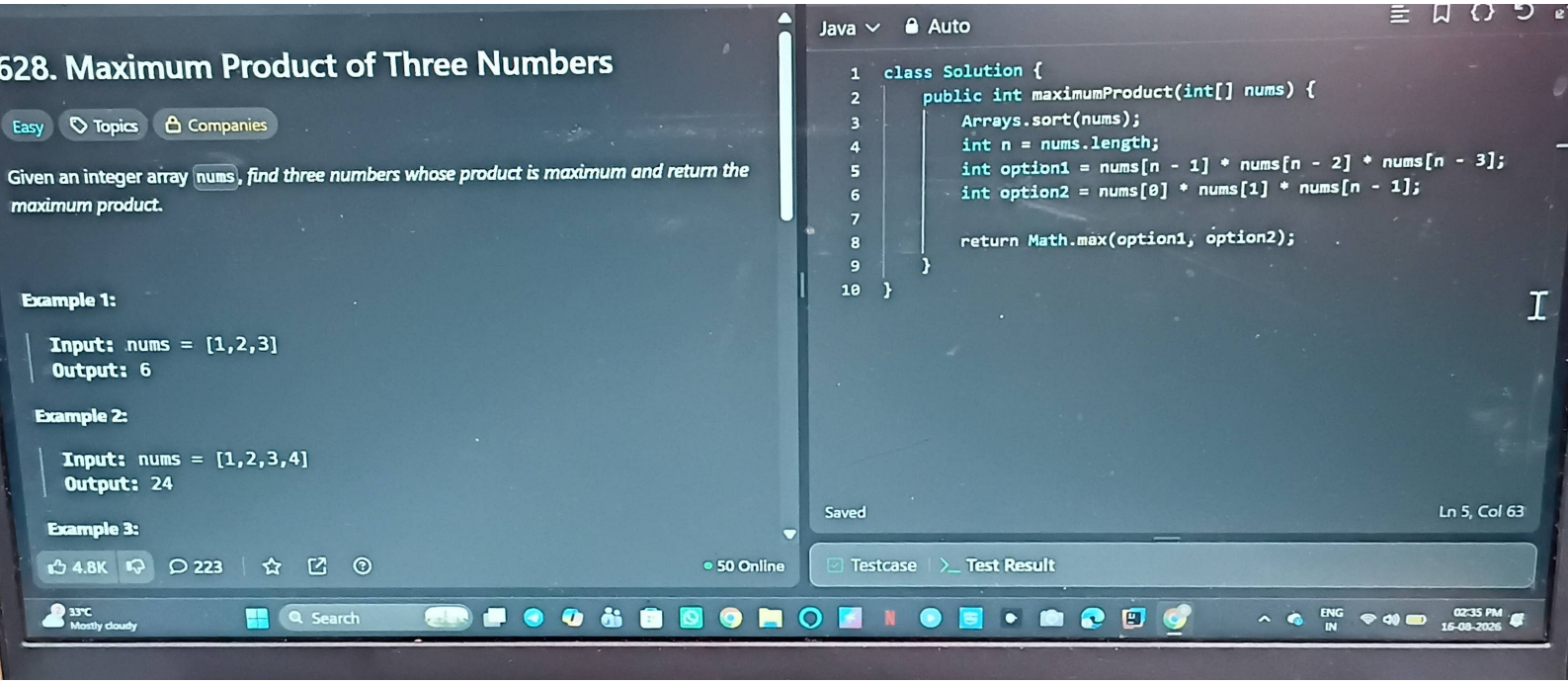

Description | Editorial | Solutions | Submissions
1. Two Sum
Solved
Easy
Topics
Companies
Hint
You are given an array of integers nums and an integer target, return indices of the two numbers such that they add up to target.
You may assume that each input would have exactly one solution, and you may not use the same element twice.
You can return the answer in any order.
Example 1:
Input: nums = [2,7,11,15], target = 9
Output: [0,1]
Explanation: Because nums[0] + nums[1] == 9, we return [0, 1].
69.6K
2.1K
1996 Online
Code | Accepted
All Submissions
10ms
20ms
30ms
40ms
Code | Java
class Solution {
public int[] twoSum(int[] nums, int target) {
for(int i = 0; i < nums.length; i++){
for (int j = i + 1; j < nums.length; j++){
if (nums[i] + nums[j] == target){
return new int[] {i,j};
}
}
View more
Write your notes here
Testcase | Test Result
33°C
Mostly cloudy
Search
ENG
IN
02:29 PM
16-08-2026

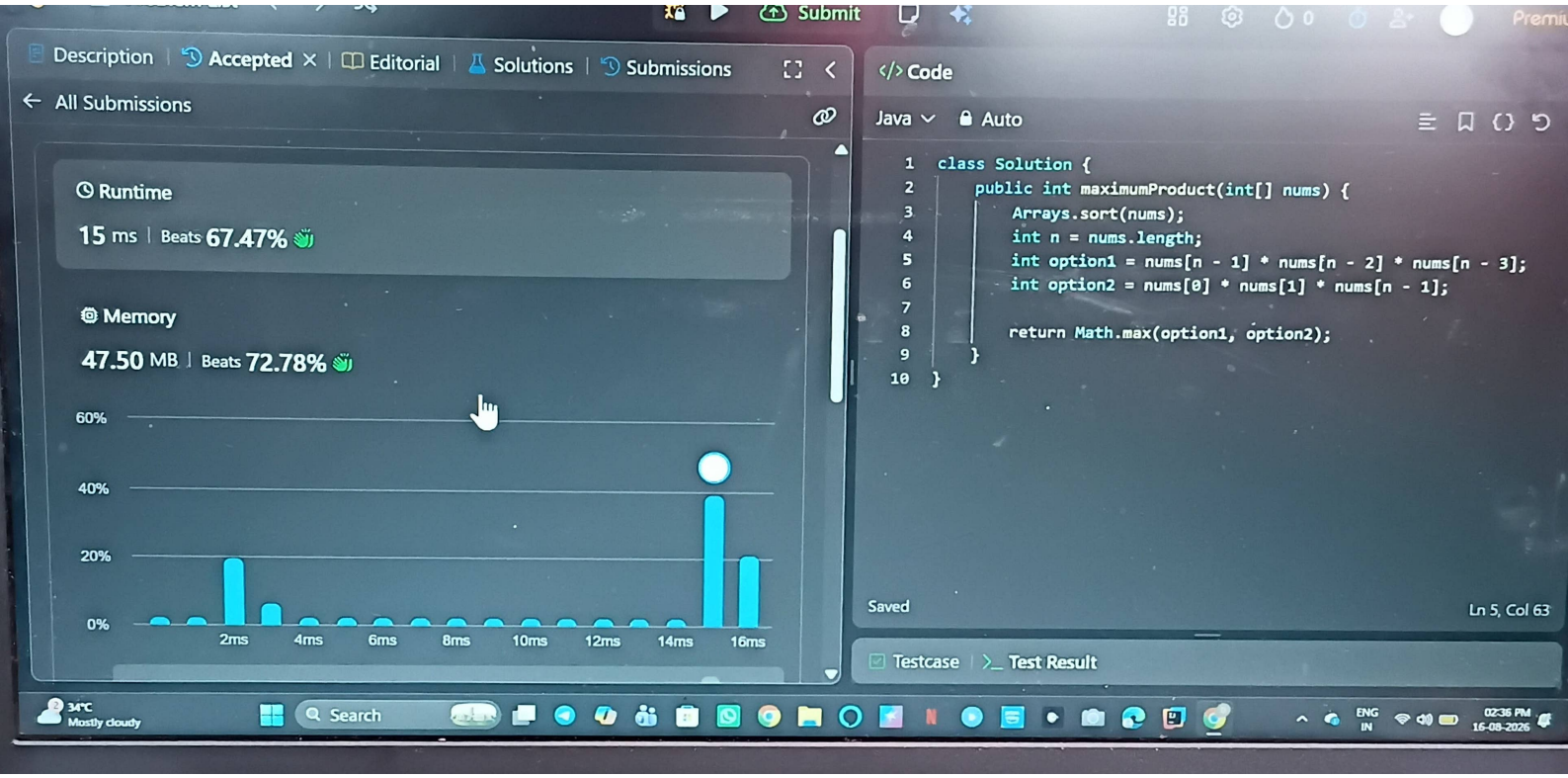

Description | Editorial | Solutions | Submissions
1365. How Many Numbers Are Smaller Than the Current Number
Easy
Topics
Companies
Hint
Given the array nums, for each nums[i] find out how many numbers in the array are smaller than it. That is, for each nums[i] you have to count the number of valid j's such that j != i and nums[j] < nums[i].
Return the answer in an array.
Example 1:
Input: nums = [8,1,2,2,3]
Output: [4,0,1,1,3]
Explanation:
For nums[0]=8 there exist four smaller numbers than it (1, 2, 2
6.1K
89
47 Online
Code
Java
Auto
1 class Solution {
2 public int[] smallerNumbersThanCurrent(int[] nums) {
3 int[] result = new int[nums.length];
4
5 for (int i = 0; i < nums.length; i++) {
6 int count = 0;
7 for (int j = 0; j < nums.length; j++) {
8 if (nums[j] < nums[i]) {
9 count++;
10 }
11 }
12 result[i] = count;
13 }
14
15 return result;
16 }
17 }
Saved
Ln 1, Col 1
Testcase
Test Result
33°C
Mostly cloudy
Search
ENG
IN
02:32 PM
16-08-2026

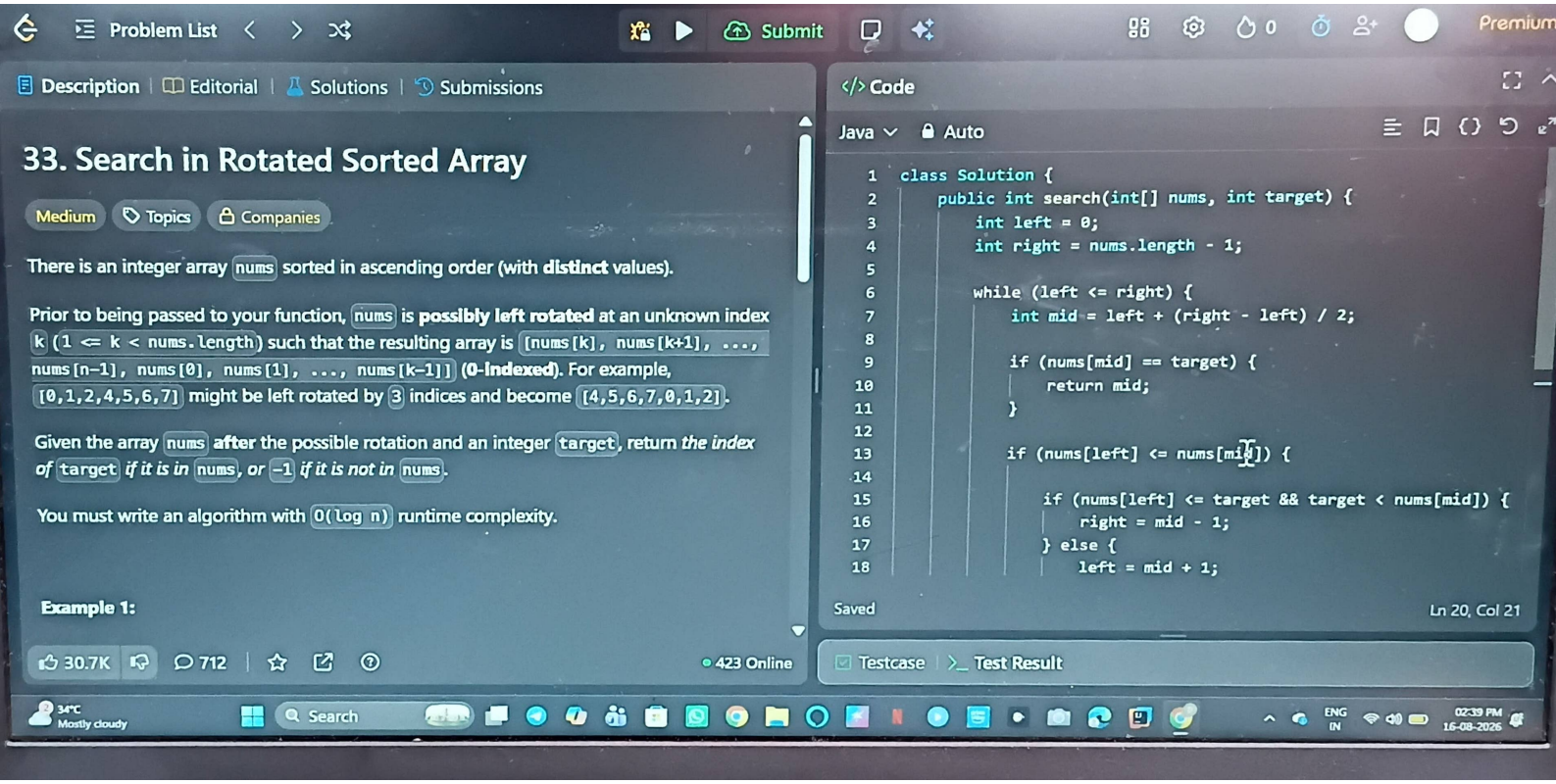

Problem List
Submit
Description
Accepted
Editorial
Solutions
Submissions
All Submissions
Runtime
7 ms | Beats 83.17%
Memory
45.28 MB | Beats 92.90%
40%
20%
0%
2ms
4ms
6ms
8ms
10ms
Code
Java
Auto
class Solution {
public int[] smallerNumbersThanCurrent(int[] nums) {
int[] result = new int[nums.length];
for (int i = 0; i < nums.length; i++) {
int count = 0;
for (int j = 0; j < nums.length; j++) {
if (nums[j] < nums[i]) {
count++;
}
}
result[i] = count;
}
return result;
}
}
Saved
Ln 1, Col 1
Testcase
Test Result
33°C
Mostly cloudy
Search
ENG
IN
02:32 PM
16-08-2026

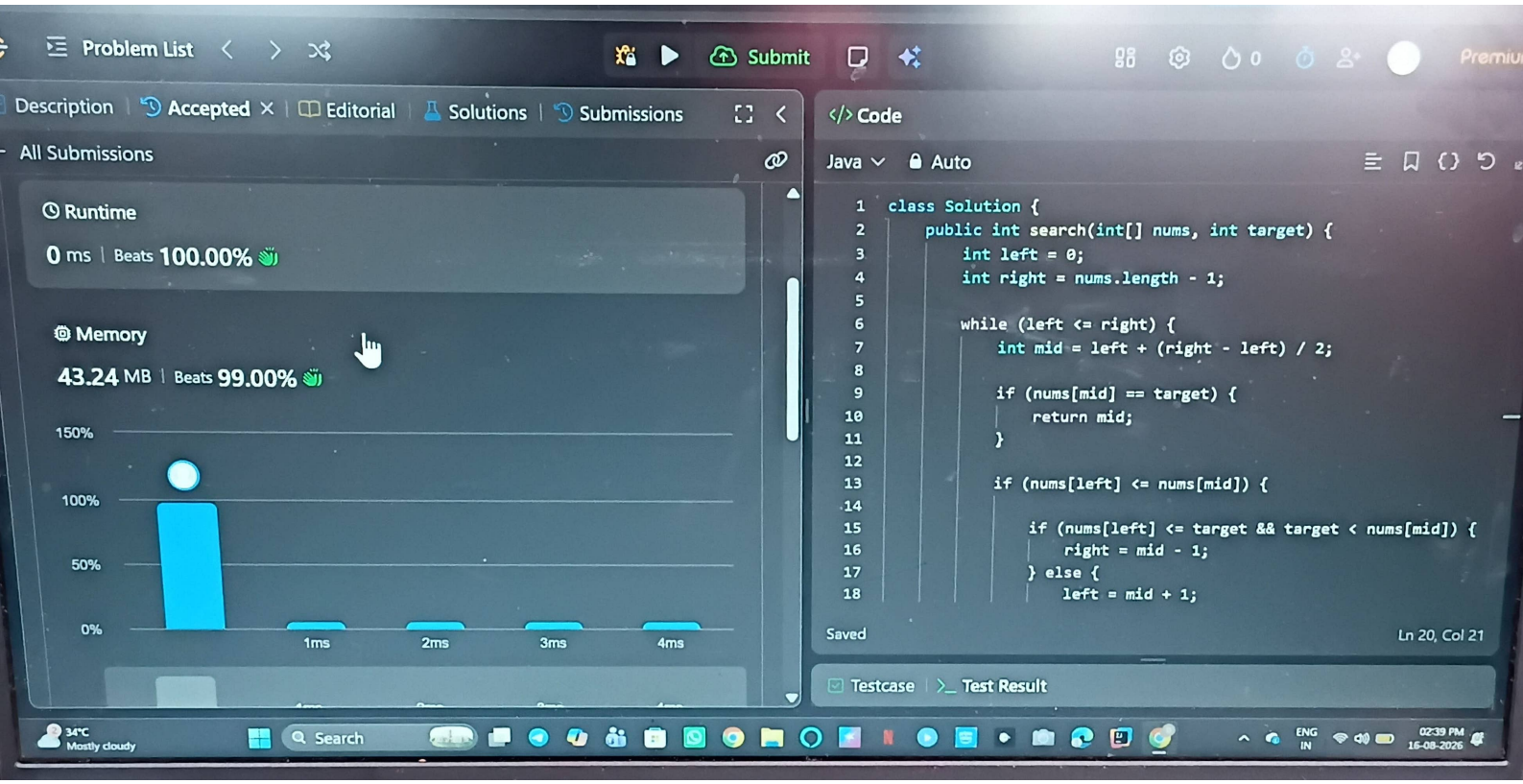

Problem List
Submit
Description
Editorial
Solutions
Submissions
26. Remove Duplicates from Sorted Array
Solved
Easy
Topics
Companies
Hint
Given an integer array nums sorted in non-decreasing order, remove the duplicates in-place such that each unique element appears only once. The relative order of the elements should be kept the same.
Consider the number of unique elements in nums to be k. After removing duplicates, return the number of unique elements k.
The first k elements of nums should contain the unique numbers in sorted order. The remaining elements beyond index k - 1 can be ignored.
Custom Judge:
The judge will test your solution with the following code:
int[] nums = [...]; // Input array
int[] expectedNums = [...]; // The expected answer with correct
19.6K
1.1K
462 Online
Code
Java
Auto
class Solution {
public static int removeDuplicates(int[] nums) {
if (nums.length == 0) return 0;
int k = 0;
for (int i = 1; i < nums.length; i++) {
if (nums[i] != nums[k]) {
k++;
nums[k] = nums[i];
}
}
return k + 1;
}
public static void main(String[] args) {
int[] nums = {1, 1, 2};
Saved
Ln 1, Col 1
Testcase
Test Result
33°C
Mostly cloudy
Search
ENG
IN
02:33 PM
16-08-2026

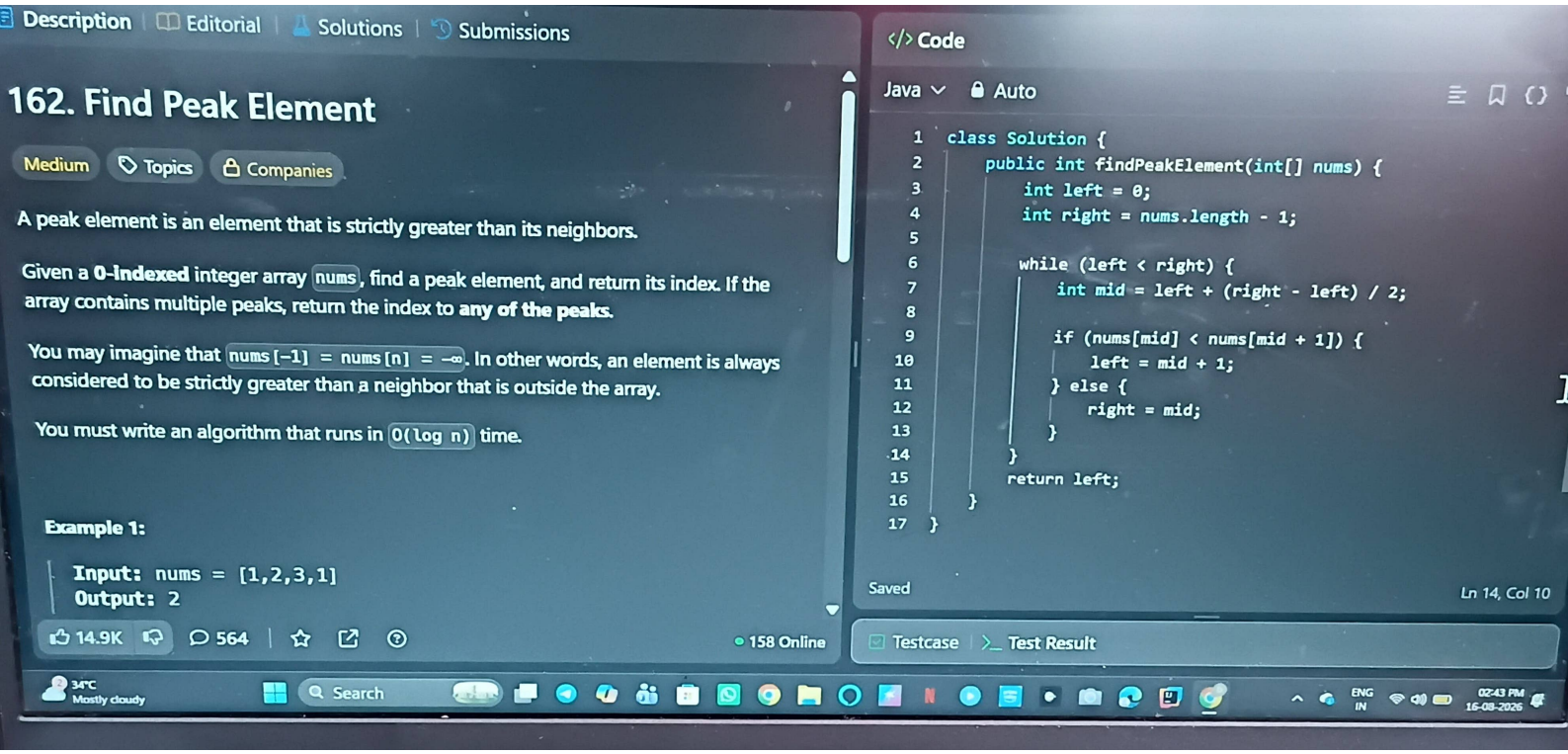

Problem List
Submit
Description
Editorial
Solutions
Submissions
26. Remove Duplicates from Sorted Array
Solved
Easy
Topics
Companies
Hint
Given an integer array nums sorted in non-decreasing order, remove the duplicates in-place such that each unique element appears only once. The relative order of the elements should be kept the same.
Consider the number of unique elements in nums to be k. After removing duplicates, return the number of unique elements k.
The first k elements of nums should contain the unique numbers in sorted order. The remaining elements beyond index k - 1 can be ignored.
Custom Judge:
The judge will test your solution with the following code:
int[] nums = [...]; // Input array
int[] expectedNums = [...]; // The expected answer with correct
19.6K
1.1K
457 Online
Code
Accepted
All Submissions
Runtime
1 ms
Beats 77.40%
Memory
46.40 MB
Beats 96.19%
75%
50%
25%
0%
1ms
2ms
3ms
4ms
0.86% of solutions used 3 ms of runtime
Testcase
Test Result
Search

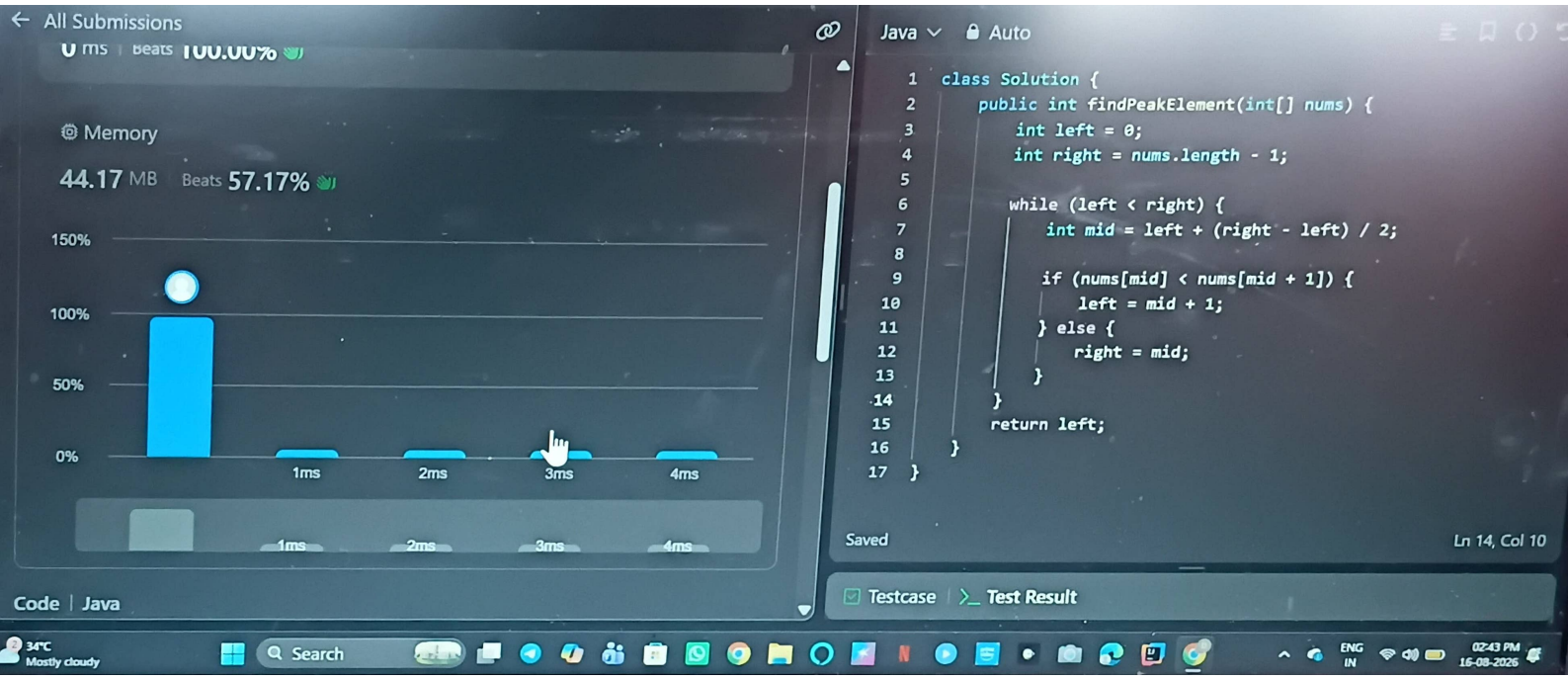

628. Maximum Product of Three Numbers
Easy
Topics
Companies
Given an integer array nums, find three numbers whose product is maximum and return the maximum product.
Example 1:
Input: nums = [1,2,3]
Output: 6
Example 2:
Input: nums = [1,2,3,4]
Output: 24
Example 3:
4.8K
223
50 Online
Java
Auto
1 class Solution {
2 public int maximumProduct(int[] nums) {
3 Arrays.sort(nums);
4 int n = nums.length;
5 int option1 = nums[n - 1] * nums[n - 2] * nums[n - 3];
6 int option2 = nums[0] * nums[1] * nums[n - 1];
7
8 return Math.max(option1, option2);
9 }
10 }
Saved
Ln 5, Col 63
Testcase
Test Result
33°C
Mostly cloudy
Search
ENG
IN
02:35 PM
16-08-2026


Submit
Description | Accepted | Editorial | Solutions | Submissions
All Submissions
Runtime
15 ms | Beats 67.47%
Memory
47.50 MB | Beats 72.78%
60%
40%
20%
0%
2ms
4ms
6ms
8ms
10ms
12ms
14ms
16ms
Code
Java
Auto
1 class Solution {
2 public int maximumProduct(int[] nums) {
3 Arrays.sort(nums);
4 int n = nums.length;
5 int option1 = nums[n - 1] * nums[n - 2] * nums[n - 3];
6 int option2 = nums[0] * nums[1] * nums[n - 1];
7
8 return Math.max(option1, option2);
9 }
10 }
Saved
Ln 5, Col 63
Testcase | Test Result
34°C
Mostly cloudy
Search
ENG
IN
02:36 PM
16-08-2026


Problem List
Submit
Premium
Description | Editorial | Solutions | Submissions
33. Search in Rotated Sorted Array
Medium
Topics
Companies
There is an integer array nums sorted in ascending order (with distinct values).
Prior to being passed to your function, nums is possibly left rotated at an unknown index k (1 <= k < nums.length) such that the resulting array is [nums[k], nums[k+1], ..., nums[n-1], nums[0], nums[1], ..., nums[k-1]] (0-indexed). For example, [0,1,2,4,5,6,7] might be left rotated by 3 indices and become [4,5,6,7,0,1,2].
Given the array nums after the possible rotation and an integer target, return the index of target if it is in nums, or -1 if it is not in nums.
You must write an algorithm with O(log n) runtime complexity.
Example 1:
30.7K
712
423 Online
Code
Java
Auto
1 class Solution {
2 public int search(int[] nums, int target) {
3 int left = 0;
4 int right = nums.length - 1;
5
6 while (left <= right) {
7 int mid = left + (right - left) / 2;
8
9 if (nums[mid] == target) {
10 return mid;
11 }
12
13 if (nums[left] <= nums[mid]) {
14
15 if (nums[left] <= target && target < nums[mid]) {
16 right = mid - 1;
17 } else {
18 left = mid + 1;
Saved
Ln 20, Col 21
Testcase
Test Result
34°C
Mostly cloudy
Search
ENG
IN
02:39 PM
16-08-2026


Problem List
Submit
Description
Accepted
Editorial
Solutions
Submissions
All Submissions
Runtime
0 ms | Beats 100.00%
Memory
43.24 MB | Beats 99.00%
150%
100%
50%
0%
1ms
2ms
3ms
4ms
Code
Java
Auto
1 class Solution {
2 public int search(int[] nums, int target) {
3 int left = 0;
4 int right = nums.length - 1;
5
6 while (left <= right) {
7 int mid = left + (right - left) / 2;
8
9 if (nums[mid] == target) {
10 return mid;
11 }
12
13 if (nums[left] <= nums[mid]) {
14
15 if (nums[left] <= target && target < nums[mid]) {
16 right = mid - 1;
17 } else {
18 left = mid + 1;
Saved
Ln 20, Col 21
Testcase
Test Result
34°C
Mostly cloudy
Search
ENG
IN
02:39 PM
16-08-2026


Description | Editorial | Solutions | Submissions
162. Find Peak Element
Medium
Topics
Companies
A peak element is an element that is strictly greater than its neighbors.
Given a 0-Indexed integer array nums, find a peak element, and return its index. If the array contains multiple peaks, return the index to any of the peaks.
You may imagine that nums[-1] = nums[n] = -∞. In other words, an element is always considered to be strictly greater than a neighbor that is outside the array.
You must write an algorithm that runs in O(log n) time.
Example 1:
Input: nums = [1,2,3,1]
Output: 2
14.9K
564
158 Online
Code
Java
Auto
class Solution {
public int findPeakElement(int[] nums) {
int left = 0;
int right = nums.length - 1;
while (left < right) {
int mid = left + (right - left) / 2;
if (nums[mid] < nums[mid + 1]) {
left = mid + 1;
} else {
right = mid;
}
}
return left;
}
}
Saved
Ln 14, Col 10
Testcase
Test Result
34°C
Mostly cloudy
Search
ENG
IN
02:43 PM
16-08-2026


All Submissions
0 ms | Beats 100.00%
Memory
44.17 MB Beats 57.17%
150%
100%
50%
0%
1ms
2ms
3ms
4ms
Code | Java
Java
Auto
1 class Solution {
2 public int findPeakElement(int[] nums) {
3 int left = 0;
4 int right = nums.length - 1;
5
6 while (left < right) {
7 int mid = left + (right - left) / 2;
8
9 if (nums[mid] < nums[mid + 1]) {
10 left = mid + 1;
11 } else {
12 right = mid;
13 }
14 }
15 return left;
16 }
17 }
Saved
Ln 14, Col 10
Testcase | Test Result
34°C
Mostly cloudy
Search
ENG
IN
02:43 PM
16-08-2026